Music Genre Classification: A Deep Learning Approach

Yunus Emre Ulusoy, Semih Doğan, Burak Ala, Alp Eren Gül, Umut Köprülü

***Abstract*—Music genre classification is a standard benchmark for audio understanding, yet real-world datasets remain challenging due to noisy labels and overlapping genre boundaries. This report studies supervised genre classification across two complementary settings: the curated GTZAN dataset and the larger, noisier Free Music Archive (FMA) collection (FMA-small and FMA-medium). We compare classical pipelines based on the official FMA handcrafted feature set (logistic regression and PCA + RBF-SVM) against spectrogram-based deep models trained on log-mel representations, including a lightweight CNN and a CNN-Transformer that aggregates multiple short excerpts per track.**

Our experiments show that the "best" approach depends on dataset properties. On GTZAN, handcrafted features paired with a nonlinear SVM achieve very high accuracy (0.919), outperforming our CNN (0.750), indicating that well-tuned classical methods can dominate in small, clean regimes. On FMA-small, learned spectrogram features become competitive: the CNN slightly exceeds the handcrafted baselines (0.591 vs. 0.580). Scaling to FMA-medium, a stronger CNN-Transformer reaches 0.692 accuracy, but Macro F1 remains substantially lower (0.394), revealing persistent difficulties on rare genres. Ablations further show that preprocessing choices—especially silent-segment filtering and normalization—significantly affect performance in noisy, long-tailed data.

I. Introduction

This report presents a study on supervised music genre classification across two complementary datasets. First, we consider the GTZAN benchmark [1], which is relatively homogeneous and widely used in prior work. Second, we evaluate the Free Music Archive (FMA) dataset [2] at two scales: FMA-small (8 genres, balanced) and FMA-medium (16 genres, strongly imbalanced). This combination allows us to analyze how model behavior changes when moving from a clean, small benchmark to a larger and more realistic long-tailed collection.

Despite its apparent simplicity, genre recognition remains challenging in realistic settings. Genre boundaries are often fuzzy, many tracks show mixed stylistic characteristics, recording conditions vary widely, and large-scale datasets commonly suffer from label noise and class imbalance. These challenges make genre classification a useful proxy task for evaluating the practical trade-offs between classical feature engineering and learned audio representations.

Preliminary results show that nonlinear models substantially outperform simple linear baselines, and that spectrogram-based CNNs can further improve performance. Unlike prior work that emphasizes architectural novelty, our study focuses on the impact of preprocessing, normalization, and data quality across datasets of varying scale.

Methodologically, we evaluate two families of approaches. (i) *Handcrafted-feature baselines*: using the official FMA feature set, we train L2-regularized multinomial logistic regression models and a PCA + RBF-kernel SVM pipeline to assess the discriminative power of classical audio descriptors. (ii) *Spectrogram-based deep models*: we train a lightweight CNN on log-mel spectrograms, as well as a segment-based CNN–Transformer architecture that extracts local spectro-temporal cues from short windows and aggregates segment-level representations using self-attention at the track level. To improve robustness in the small-data regime, we apply waveform- and spectrogram-level data augmentation, investigate normalization strategies, and filter silent segments to avoid training on low-information excerpts.

The report concludes with a discussion of current limitations related to dataset size, model capacity, and data augmentation, and outlines future directions toward more robust deep learning systems for music genre classification.

Overall, the main contribution of this work is a unified empirical comparison across dataset regimes, highlighting how preprocessing, normalization, and dataset quality interact with model choice—from strong classical baselines to compact deep architectures—under limited labeled data and severe class imbalance.

II. Literature Review

The classic GTZAN work by Tzanetakis and Cook (2002) [1] is the seminal study that introduced the GTZAN Music Genre dataset and established the modern formulation of the music genre classification problem. This work proposed three families of handcrafted audio features—timbral texture (e.g., MFCC-derived cues), rhythmic content (beat and periodicity), and pitch histograms—computed over 30-second tracks. The authors evaluated statistical pattern-recognition classifiers in two modes: whole-file (offline) and real-time frame-wise. Their reported accuracies were 0.61 for the whole-file case and 0.44 for real-time classification.

In terms of methodology, they first define a timbral texture descriptor computed on short-time frames of roughly 23 ms (512-sample windows) using a Fourier transform, which they derive spectral centroid, spectral rolloff, spectral flux, zero-crossing rate, and Mel-frequency cepstral coefficients. Then they integrate these frame-level measures over a 1-second "texture window" by taking means and variances. Rhythmic content is extracted by passing the audio through a wavelet transform into octave-spaced sub-bands. After the process, they take the amplitude envelopes for pitch content, and

they adapt the multipitch detection method of Tolonen and Karjalainen, splitting the signal into low and high bands. Those are combining half-wave rectified, and low-pass filtered envelopes, and again using an enhanced autocorrelation to emphasize fundamental periods while suppressing simple multiples, accumulating the salient lags into pitch histograms in a MIDI-pitch space and then folding them into pitch-class histograms. In the whole-file mode, they compute rhythm and pitch features over the entire 30-second excerpt and average timbral texture vectors across all texture windows.

They treat genre recognition explicitly as a statistical pattern-recognition problem and train several classic machine learning classifiers on their handcrafted feature vectors. In particular, they use simple multivariate Gaussian classifiers (one Gaussian per genre) and Gaussian Mixture Model (GMM) classifiers whose parameters are learned with the Expectation Maximization algorithm. In addition, they evaluate a non-parametric k-Nearest Neighbors (k-NN) classifier operating directly in the 30-dimensional feature space, using distance to stored training examples rather than an explicit probability model. Together, these methods let them compare generative density models (Gaussian, GMM) and instance-based methods (k-NN) on the same GTZAN feature representation.

Bergstra and colleagues (2006) [3] segment each 30-second clip into short windows, compute MFCC and related low-level descriptors per window, and then aggregate them to the track level by taking statistics such as mean, variance, and sometimes higher moments across time. These vectors are fed into an AdaBoost. MH is a multi-class and hamming loss, which is used in AdaBoost. An MH ensemble that builds many shallow decision trees, each focusing on examples that were misclassified by the previous trees, and gradually forms a strong classifier that defines a complex nonlinear decision boundary in the feature space. The boosting procedure implicitly performs a kind of feature selection, because only features that help reduce error are repeatedly used in the weak learners, and this is one reason the method pushes GTZAN accuracies into the low eighty percent range.

Beyond this basic description, Bergstra makes the aggregation and boosting strategy very explicit: each 30-second GTZAN excerpt is first sliced into short, fixed-size frames of 46.44 ms (1024 samples at 22 050 Hz), from which they compute several families of low-level descriptors, including truncated FFT coefficients, real cepstral coefficients (RCEPS), Mel-frequency cepstral coefficients (MFCC), linear prediction coefficients (LPC), zero-crossing rate, and simple spectral shape measures such as spectral centroid, spread, and rolloff. These frame-level feature vectors are then grouped into non-overlapping segments consisting of m consecutive frames (they systematically explore $m \in \{40, 50, 75, 100, 150, 300, 600\}$, corresponding to segment lengths from roughly 1.8 s up to about 28 s), and for each segment they "aggregate" by fitting independent one-dimensional Gaussians to every feature dimension and taking the sample mean and variance as the segment-level representation, so that each segment is turned into a single, high-dimensional vector of mean/variance statistics rather than a long time series. Each such aggregated segment inherits the genre label of its parent song and becomes one training example for a multiclass AdaBoost.MH classifier, so the effective training set size grows with the number of segments per song, while the classifier still only ever sees fixed-length vectors. In the AdaBoost.MH stage, the input to each weak learner is a d-dimensional segment vector $x \in \mathbb{R}^d$, and the weak learner returns a vector-valued prediction $h^{(t)}(x) \in \{-1, +1\}^K$ over the K genre classes, where each component indicates whether this weak rule "votes for" or "votes against" a given class, and the strong classifier is the weighted sum

$$g(x) = \sum_{t=1}^{T} \alpha^{(t)} h^{(t)}(x),$$

with the final decision taken as the class index with the largest component of $g(x)$. As weak learners, they use either decision stumps (one-level trees with a single threshold on one feature dimension) or small decision trees; the stump-based variant is denoted AB. Stump and the shallow-tree variant AB. Tree, and both are trained inside AdaBoost.MH by reweighting training examples after each iteration so that misclassified segments receive a higher weight in the next weak learner. Because each stump depends on only a single feature dimension, running AdaBoost.MH with T boosting rounds effectively selects at most T "useful" features out of the full aggregate vector; when $T \ll d$, this acts as an embedded feature-selection mechanism where uninformative or redundant MFCC/FFT/LPC statistics are simply never picked by any weak learner. They also derive the computational complexity of this boosted ensemble: assuming feature values are pre-sorted, training the weak learner on n segments with d features and K classes costs $\mathcal{O}(nKd)$ per round, and the full AdaBoost.MH procedure runs in $\mathcal{O}\big(nd(KT + \log n)\big)$ time, which scales linearly in the number of segments for fixed T and is therefore more tractable on large music collections than SVMs, whose training cost is quadratic in the number of training points. At test time, each segment of a song is classified independently by the boosted ensemble, and the segment-level discriminant scores $g(x)$ are combined (e.g., averaged or summed) across all segments belonging to a song to produce a final song-level genre decision, which is the metric reported when they quote GTZAN accuracies. Their systematic segment-length experiments show that segment-level accuracy tends to increase with longer segments, but song-level accuracy first rises and then falls, with an intermediate length around 3.5 s (75 frames) giving the best trade-off between having enough frames to make aggregated means/variances reliable and having enough segments per song to enlarge the training set, and it is precisely in this regime that the AdaBoost.MH ensemble with aggregate MFCC/RCEPS/FFT features reaches GTZAN accuracies in the low 80% range. Compared against alternative classifiers (feed-forward neural networks and SVMs) trained

on the same aggregated feature sets and segmentations, the AdaBoost.MH ensembles AB.Stump and AB. Trees are consistently among the top performers, especially when using MFCC or RCEPS aggregates, which reinforces the core methodological point of the paper: for GTZAN-style genre recognition, combining simple frame-level audio descriptors into segment-wise mean/variance aggregates and feeding them into a large-margin boosting ensemble is an effective and computationally efficient way to implement a highly non-linear decision rule in a handcrafted feature space.

A notable CNN study on the GTZAN dataset was done by Zhang (2016) [4]. STFT magnitude spectrograms are used as input and propose two main ideas: (i) combining max pooling and average pooling to get richer statistics from each feature map, and (ii) adding a shortcut (residual) connection to make the network easier to train. The audio is cut into 3-second clips with 50% overlap, and predictions are then combined at the song level. With 10-fold song-level cross-validation on GTZAN, their best model reaches about 87.4% accuracy, which is clearly higher than earlier handcrafted feature methods that usually stay between 60% and 80%. They define two architectures called nnet1 and nnet2. Both take as input a 3-second STFT magnitude spectrogram with around 128 time frames and 513 frequency bins. This 2D time–frequency matrix is treated like an image and passed through several convolutional layers followed by fully connected layers to get the final 10-class genre output. In nnet1, the feature extractor has three convolutional layers. The first layer uses 128 kernels of size 4×513 that move along the time axis, so each kernel sees about 127 ms and all frequencies, and can learn short spectro-temporal patterns. The second and third layers use kernels of size 4×1, with 128 and 256 feature maps, which increases the temporal receptive field and helps the model learn more abstract patterns such as rhythm fragments or slowly changing timbres. After each convolution, except the last one, they apply a 2×1 max-pooling layer along time. This reduces the time resolution, adds some time-shift invariance, and lowers the computational cost while keeping the strongest activations.

A key step in nnet1 is what happens after the third convolutional layer. At this point, they apply global pooling over the time dimension in two ways: global max pooling and global average pooling, with a window equal to the remaining time length (26×1 in nnet1). Then they concatenate the two pooled outputs. In this way, the final feature vector contains both the maximum activation and the mean activation for each feature map. Their experiments show that this combined pooling works better than using only max pooling or only average pooling. This fixed-length vector is then fed into three fully connected layers with 300, 150, and 10 units. The hidden layers use ReLU activations, and the last layer uses a softmax to produce probabilities for the 10 GTZAN genres.

The second architecture, nnet2, uses the same general pipeline but adds a residual-style shortcut connection. The output of the first convolutional layer is added directly to the output of the third convolutional layer. This means the third layer learns a residual function F(x), and the final representation is x + F(x), similar to a small ResNet block. This helps optimization and makes training the deeper network easier. In nnet2, all three convolutional layers have 256 feature maps, with kernels of size 4×(full-frequency or 1). Before the element-wise addition, they use zero-padding in time so that the output of the first layer (125 time steps) and the output of the third layer (119 time steps) have matching time dimensions. After the residual block, they again apply global max pooling and global average pooling over time, concatenate the results, and use the same 300-150-10 fully connected classifier as in nnet1. So the main differences between nnet1 and nnet2 are the residual shortcut and the slightly different number of feature maps. For training, both networks use Adadelta with the default learning rate of 1.0 and categorical cross-entropy loss. They train with mini-batches of size 50 and use ReLU activations in all convolutional and dense layers. Dropout with a rate of 0.2 is applied to reduce overfitting. In nnet2, they also apply batch normalization, which stabilizes the distributions of activations and allows faster and more stable training, especially with a relatively high learning rate. On the data side, each 30-second GTZAN track is split into overlapping 3-second clips with 50% overlap. For each clip, they compute an STFT with 1024-sample windows and 50% overlap, and they use the magnitude spectrum (513 frequency bins) as input to the network. They report that this clip-based setup gives better performance than using a single full-song spectrogram.

At test time, the network produces a probability vector for each 3-second clip. For a given song, they sum the probability vectors of all clips from that song and then choose the genre with the highest summed probability as the final song-level prediction. All reported results, including the 87.4% best accuracy, use 10-fold song-level cross-validation, which avoids having clips from the same song in both training and test sets.

Their ablation studies clarify which design choices matter. For nnet1, using only average pooling gives 84.4% accuracy, which is already better than only max pooling (79.9%). When they concatenate max and average pooling, accuracy increases slightly to 84.8%, which suggests that max and mean activations provide complementary information. For nnet2, the residual connection makes the base model stronger: with only max pooling, nnet2 already reaches 85.0%. When they again combine max and average pooling in nnet2, accuracy rises to 87.4%, which is their best result and higher than all previous GTZAN models they compare with.

They also compare their networks to earlier deep learning methods, such as k-max CNN features plus SVM (83.9%), deep neural networks on FFT-based aggregate features (83.0%), and multi-layer invariant representations on log-STFT (82.0%). Their best CNN, which uses STFT magnitude spectrograms, a residual shortcut, and combined global pooling, achieves the highest GTZAN accuracy among these approaches. Overall, the work shows that CNNs on spectrogram “images” can benefit from combining global max and

average pooling, and that adding even a single residual block can noticeably improve training and generalization on a small, noisy dataset like GTZAN. For this reason, the Zhang et al. architecture is often used as a strong reference model in later CNN-based music genre classification studies.

In this article, Liu et al. [5] used the GTZAN dataset, which includes 1000 audio clips from 10 musical genres. They introduced a new approach by using a transformer-based model instead of classical CNN layer models. The authors do not use any CNN layers. They mainly rely on self-attention to learn audio features' global relationships. For instance, CNNs have problems utilizing every layer's feature data, and RNNs lose the beginning of the data, but with transformers, the model can attend to all time steps and capture patterns that carry music's characteristics.

The model's main method is to extract MFCC-based feature sequences from the audio, then feed these sequences into a Transformer encoder. Positional embeddings are added to preserve temporal order, and a special classification token is used so that the Transformer can summarize the entire sequence for genre prediction.

As a result, this Transformer-based model achieves higher accuracy than average CNN models on the GTZAN dataset, showing that global self-attention modeling is effective for music genre classification.

In another article, Shen et al. [6] used the FMA-small dataset, which consists of 8000 audio clips from 8 different music genres. In this project, they presented a new modeling approach based on Functional Data Analysis with continuous mathematical functional features. The authors do not use classical CNN components like Conv2D or MaxPooling with spectrograms. Instead of those, they used basis expansions and functional representations. These functional representations also help them perform better on small, noisy datasets.

As a final result of this model, they reached 68-72% accuracy with the FNN model for the FMA-small dataset. This model outperformed classical Machine Learning models and also showed similar results to simpler CNN models.

In this article, Zhuang et al. [7] used the GTZAN dataset, which includes 1000 samples from 10 different genres. They classified this dataset with their newly developed CNN model, which includes different features like multi-scale convolution, broadcast-based skip connections, dense connections, and feature fusion.

As a short summary of these methods, in multi-scale convolution, they use three different kernels for the same layer. All these kernels hold different music features such as high and mid frequencies, bass, rhythm, etc. Broadcast-based skip connections help upper layers hold lower layers' features; as a result of this, we preserve lower-layer data. Dense connections provide every layer with the ability to connect to get better gradient flow. Feature fusion combines all layers' features; normally, CNNs only use higher-level layer features.

As a result of the BBNN model, they increased accuracy by almost 4–5%, from 89–91% to 93.9% compared to CNN models, and also performed better than CRNN models by about 2–3%. They utilize mel-spectrogram, Conv2D, ReLU activation, Batch Normalization, MaxPooling, Dropout, Softmax classifier, SGD/Adam optimizer usage, cross-entropy loss, and mini-batch training. These parts are similar to classical CNN models. They also used an 80–10–10 train–test–validation data separation to train the model.

In their article, Ndou, Ajoodha, and Jadhav [8] used the GTZAN dataset with 1000 audio excerpts. They are distributed across 10 genres, so there will be 100 songs per genre. The study was done in 3 phases. In Phase A, they extracted the features using MFCC aggregation, spectral features, temporal statistics, chroma features, and beat histograms. In Phase B, they focused on spectral contrast, rolloff, MFCC means, RMS energy, and beat histogram features. In Phase C, they used a modified version of GTZAN, which consisted of 10000 excerpts of 3-second duration. They included features such as chroma, RMS, spectral features, zero-crossing rate, MFCCs, harmony, and tempo. They used traditional machine learning models such as kNN, Linear Logistic Regression, Multilayer Perceptron, Random Forests, Support Vector Machines, and Naive Bayes. They used a CNN with 5 convolutional blocks. Filters were 3×3. Activation function used is ReLU, and they used 2×2 max pooling with 0.2 dropout rate. CNN is trained on spectrograms and 20 MFCCs.

In Phase A, there were 459 30-second features. Linear Logistic Regression performed the best with 81% accuracy. Random forests had 75.70% accuracy, SVMs had 75.40% accuracy, MLP had 75.20% accuracy, kNN had 72.80% accuracy, and Naive Bayes had 53.20% accuracy. In Phase B, there were 54 30-second features. Here, SVM performed the best with 80.80% accuracy, MLP had 77.30% accuracy, Logistic Regression had 75.80% accuracy, Random Forests had 72.40% accuracy, kNN had 69.70% accuracy, and Naive Bayes had 54.50% accuracy. In Phase C, kNN made a remarkable improvement and performed with 92.69% accuracy. This was due to a couple of reasons. First, since we increased the amount of data by dividing it into 3-second segments, the model was trained better. Also, according to the article, 30-second segments could vary, but 3-second segments can be more precise indicators of the genre. This increase in accuracy is also due to kNN's nature, because abundant data is specifically better for kNN. On the other hand, CNN had an accuracy of 72.40%. This was due to the training data size. Even though they divided the songs into 3-second segments, this is still a small size for a deep learning algorithm. Another reason is that only 5 convolutional layers are used, which makes the algorithm pretty basic. Also, using 50 epochs for 3-second features may have caused the model to overfit.

The implication of this article is the following: Traditional machine learning approaches can exceed deep learning approaches depending on the dataset and quality of feature engineering. So if the sample size is not too large, implementing well-crafted features instead of creating deep and sophisticated learning algorithms could be a better choice.

In their article, Oramas et al. used a dataset they constructed called "MuMu Dataset" [9]. It consists of 31471 albums and 147295 songs. But what is different in this dataset is that they also have the album cover image, album reviews, and text reviews. What makes this research stand out from others is the fact that it uses audio-based, text-based, and image-based modalities. They also emphasize that combining them achieves better scores in the end.

In the Audio-Based Deep Learning Approach, as input, they used log-compressed CQT spectrograms with 96 frequency bins and 15-second patches. For the CNN architecture, they tested various models with filter sizes 3×3, 4×96, and 4×70. The model has 4 convolutional layers and a max-pooling layer after each. For the output, they implemented 2 configurations. Sigmoid and Cosine. The cosine approach performed consistently better than the Sigmoid because it took the relations between genres into account instead of treating each of them as fully independent genres. In the Text-Based Deep Learning Approach, they took all the reviews of the album and created a single text. They utilized the Vector Space Model in this approach. They also applied a method where they map words to Wikipedia entries and utilize these Wikipedia categories to classify the song. Text performed better than audio with a 0.917 AUC compared to 0.888 AUC because customer reviews are more explicit and signal some cultural meanings that audio cannot convey directly. In the Image-Based Deep Learning Approach, they used ResNet. It was initialized with ImageNet weights. However, this method only had 0.743 AUC.

The significance of this research is the following: Audio is not the most important source for classifying genres because the text-based approach performed way better than audio. So we understand user reviews, tags, mentioning cultural references in the song, etc. can really help classifying the classifying the fact that combining all these 3 models gave us the best performance (0.916 AUC) shows us that different modalities of the song, such as text, audio, and image, complement each other, and they all contribute to something that the others can't.

In his article, Bahuleyan [10] utilized the AudioSet dataset. It consists of 40540 10-second audio clips. They selected 7 genre categories: 8100 Pop music clips, 7990 Rock music clips, 6958 Hip Hop music clips, 6885 Techno clips, 4247 Rhythm and Blues clips, 3363 Vocal music clips, and 2997 Reggae music clips. But the problem with the dataset was that these data are not raw audio clips; they were basically YouTube videos with start and end time stamps. So, researchers had to develop a method for creating the actual raw data. Some YouTube videos included background noise, sound effects, and non-musical audio elements. So some videos were excluded, possibly creating a sampling bias.

The architecture they used for the deep learning is VGG-16. VGG-16 consists of 13 convolutional layers and 3 fully-connected layers, and 3×3 convolutional filters throughout all layers. A total of 5 convolutional blocks are used, and each block contains a max pooling operation. The number of filters is 64 in the first block, then it is doubled after every block, becoming 512 at the final block. So that model can learn more abstract features as it goes through the deeper layers. They used Adam Optimizer, and models were trained with 10 epochs and a batch size of 32. Also, it should be noted that accuracy was maximum at epoch 4, then it started to decrease, indicating the possibility of overfitting. They used 2 methods to prevent the model from overfitting: They used L2 regularization and added a penalty so that really large weights get penalized. They also used dropout regularization with a dropout rate of 0.3. In the traditional approach, they separated features into two subcategories: time domain features and frequency domain features. Time domain features included mean, standard deviation, skewness, kurtosis of amplitude, Zero Crossing Rate, and Tempo (97 features in total). Frequency domain features included MFCCs, 12 Chroma features, Spectral Centroid, Spectral Bandwidth, etc. The learning methods tested are Logistic Regression, Random Forest, XGBoost, and SVM with RBF kernel. As a result, the VGG-16 transfer learning method had 0.63 accuracy, surpassing the best traditional method, XGBoost, with a 0.59 accuracy.

This article's significance is the following: In really large data sizes, like 40000+ here, CNNs with transfer learning outperform traditional machine learning approaches. A significant number lies around 10000 samples for the GTZAN datasets and 40000 for AudioSet.

In their article, Pelchat and Gelowitz [11] used two different music libraries. The first library consisted of 1880 3-minute songs. This dataset was a bit unbalanced because there were 630 Pop songs, but for example, only 27 Electronic songs. The model was biased toward the pop songs and didn't give a good performance on genres with a small amount of data. For data preprocessing, first, they converted the stereo sound to mono sound. Then, they used the SoX command-line to create spectrograms. Each 3-minute song sample is divided into 128 pixel spectrogram segments so that each segment represents 2.56 seconds. So even though the number of songs is very small, the data for training the model was abundant. The 2nd dataset was more balanced. This version consisted of around 200 songs for each genre, which is around 15000 spectrogram pieces per genre. 70% split for training, 20% for validation and 10% for test. In the CNN architecture, they used 4 convolutional layers, but later expanded to 6 layers. Also, they used 2×2 kernels with a stride of 2, and max-pooling is used after every layer. For initialization, Xavier is used, and for activation, ReLU is used. For regularization, a dropout rate of 0.5 is used. CNN architecture also includes a 4096-unit fully connected layer, and it is trained using the Adam Optimizer.

This study's significance comes from improving the overfitting situation by changing some parameters. They started with a 95% validation accuracy and a 47% test accuracy for 995 songs of 27 genres. Later, they reduced this to 7 genres with 1880 songs and got 62% accuracy. Also, they observed that changing the activation function from ELU to

ReLU made the accuracy increase by 5%, accuracy becoming 67%. Also, they switched from Hann to Hamming window for creating spectrograms, and accuracy became 71%. Overall, besides making major changes to parameters and algorithms, a balanced pick of genre distribution and sample size can improve the model's accuracy.

Beyond pure deep learning models, there are also hybrid and optimization-based pipelines for GTZAN. A recent example is the VMD-IWOA-XGBOOST method by Gan (2024) [12], published in MDPI Mathematics. In this work, they first extract features with Librosa, then apply Maximal Information Coefficient (MIC)–based feature selection, and finally use Variational Mode Decomposition (VMD). Both the VMD step and the XGBoost classifier are tuned with an Improved Whale Optimization Algorithm (IWOA). On the GTZAN dataset, this pipeline reaches 85.5% accuracy, 85.4% macro-precision, 86.6% macro-recall, and 85.5% macro-F1. These balanced scores show that hybrid optimization pipelines can be very competitive for music genre classification. In more detail, they design a multi-stage signal processing and optimization pipeline. First, for each GTZAN track, they use Librosa to compute common time-domain and frequency-domain features, such as zero-crossing rate, spectral centroid, spectral roll-off, spectral bandwidth, chroma features, root-mean-square (RMS) energy, delta features, Mel-spectrogram statistics, tempo, and MFCCs. This turns every 30-second audio file into a feature matrix that reflects both short-term timbre and longer-term changes in the signal. On this feature set, they apply the Maximal Information Coefficient (MIC) to measure how strongly each feature (possibly in a nonlinear way) is related to the genre label. MIC is computed by scanning over different 2D grids on the joint distribution of a feature and the labels, then taking the maximum normalized mutual information. The highest-scoring features are kept, and the top five are sent to the next stage.

These top features are then processed by Variational Mode Decomposition. Here, each selected feature is treated as a 1D time series. VMD tries to represent it as a sum of several band-limited components called intrinsic mode functions (IMFs), each capturing a different time scale. This is formulated as a constrained variational problem that uses Hilbert transforms, frequency shifting, and an augmented Lagrangian solved with an alternating direction method of multipliers. The quality of this decomposition depends a lot on hyperparameters, especially the number of modes K and the quadratic penalty factor alpha. To tune these, the authors use their Improved Whale Optimization Algorithm, with a fitness function that reflects both the quality of the decomposition and the final classification performance. The IWOA is an extension of the original Whale Optimization Algorithm. It changes the algorithm in three main ways. First, it adds an adaptive inertia weight that changes with the iteration number, which helps the algorithm explore more in the beginning and exploit more near the end. Second, it replaces the fixed logarithmic spiral used in the encircling phase with a variable helix whose shape changes over time, so the search paths become more diverse. Third, it introduces a differential variance factor that slightly perturbs solutions around the current best one, which helps escape local optima, especially in later iterations when the standard WOA would normally get stuck. After VMD decomposes each selected feature with IWOA-tuned K and alpha, the resulting decomposed features are split into training and test sets with an 80/20 split. These are then used as input to an XGBoost classifier, whose own hyperparameters are again optimized by IWOA. For XGBoost, they start from the usual gradient-boosted decision tree framework. The objective function combines a differentiable loss term with a regularization term that controls tree complexity. This objective is approximated with a second-order Taylor expansion in the tree scores, which is the standard XGBoost approach. IWOA is used to search over key hyperparameters such as the number of trees, maximum depth, learning rate, and gamma (the regularization for splits) to maximize validation performance. In this way, the same metaheuristic optimizer jointly tunes both the signal decomposition (VMD) and the classifier (XGBoost), trying to find a globally good match between the features and the model. In their experiments, Gan et al. compare the full VMD-IWOA-XGBOOST pipeline to a wide range of baselines on GTZAN, all using the same Librosa feature set for fairness. The baselines include plain AdaBoost, a back-propagation neural network, an LSTM, gradient boosting decision trees, random forests, standard XGBoost, and a simpler WOA-XGBoost variant. The results show a clear ranking: vanilla XGBoost already beats many traditional models with about 0.665 accuracy and 0.665 macro-F1. WOA-tuned XGBoost improves this to around 0.765 accuracy and 0.767 macro-F1. When they also add VMD and switch from WOA to IWOA, they get the best scores: 0.855 accuracy, 0.854 macro-precision, 0.866 macro-recall, and 0.855 macro-F1. Since macro metrics average across all classes, these results mean the hybrid model helps not only the majority genres but also the minority or harder genres. They also run 10-fold cross-validation and t-tests to show that the improvements of VMD-IWOA-XGBOOST over all baselines are statistically significant. At the same time, they observe that it is actually cheaper to run than the simpler WOA-XGBoost, because the improved optimizer converges faster. Overall, the study shows that combining feature selection, adaptive signal decomposition, and metaheuristic hyperparameter tuning around a strong but standard model like XGBoost can rival or even beat many deep models on GTZAN, while keeping the pipeline modular.

III. Methodology

A. Datasets

We conduct experiments on two datasets:

1) Free Music Archive (FMA): The FMA dataset [2] is a large-scale collection of Creative Commons-licensed music. We use FMA-small and FMA-medium:

- **FMA-small**: 8,000 tracks, 8 balanced genres (Electronic, Experimental, Folk, Hip-Hop, Instrumental, International, Pop, Rock)

- **FMA-medium**: 25,000 tracks, 16 genres (adds Blues, Classical, Country, Easy Listening, Jazz, Metal, Old-Time/Historic, Soul-RnB, Spoken)

All audio is sampled at 22,050 Hz. We use the official FMA train/validation/test splits provided in the metadata to ensure reproducibility and fair comparison with prior work. FMA represents a challenging, realistic scenario with noisy labels, diverse production quality, and long-tailed class distributions.

2) GTZAN: The GTZAN Music Genre dataset [1] is a widely-used benchmark consisting of 1,000 audio excerpts (100 per genre) across 10 genres: Blues, Classical, Country, Disco, Hip-Hop, Jazz, Metal, Pop, Reggae, and Rock. Each track is 30 seconds at 22,050 Hz.

GTZAN is valuable because it is widely reported in previous work and provides a more homogeneous setting, allowing evaluation under cleaner and more controlled conditions. Finally, GTZAN is relatively small compared to FMA, which allows analysis of how classification accuracy varies with the size of the data set.

Stratified 80–10–10 splits are used for the GTZAN experiments to ensure balanced class distributions between splits.

B. Audio Representation

1) Log-Mel Spectrograms: Audio tracks are converted to log-mel spectrograms with the following parameters:

- **Mel bands**: 128
- **FFT size**: 1024 or 2048 (for CNN experiments)
- **Hop length**: 512
- **Power**: 2.0 (power spectrogram)
- **Conversion**: Amplitude to dB scale

Full-track spectrograms are cached as float16 tensors. Given waveform $x[t]$, we compute an STFT magnitude spectrogram $|X(f,\tau)|$ and map it to mel scale:

$$M(m,\tau) = \sum_f H_m(f)\,|X(f,\tau)|^2,$$

where $H_m(\cdot)$ is the m-th mel filterbank. We then apply log compression (dB scale):

$$S(m,\tau) = 10\log_{10}(M(m,\tau)+\epsilon).$$

2) Segmentation Strategy: For CNN–Transformer experiments, each track is divided into 8, 12, or 14 segments, with each segment lasting 3 or 5 seconds.

- **Training**: Random segment start positions (data augmentation)
- **Validation/Test**: Deterministic equally-spaced positions (reproducibility)

C. Normalization Strategies

We employ three normalization approaches:

1) **Per-sample (local) normalization**: Each spectrogram is independently z-score normalized using its own statistics:

$$x_{\text{norm}} = \frac{x - \mu_{\text{sample}}}{\sigma_{\text{sample}}}$$

2) **Global normalization**: All spectrograms are normalized using dataset-wide statistics computed from 2,000 training tracks:

$$x_{\text{norm}} = \frac{x - \mu_{\text{global}}}{\sigma_{\text{global}}}$$

3) **No normalization**: Raw dB-scale spectrograms (baseline comparison)

D. Data Augmentation

During training, we apply aggressive data augmentation to improve generalization:

- **Waveform-level**: Random time shifting (±10%), random gain (±6 dB), additive Gaussian noise ($\sigma = 0.003$)
- **Spectrogram-level**: SpecAugment with 2 time masks (param=24) and 2 frequency masks (param=12)

For a fair comparison, validation, and test data are not augmented.

E. Model Architectures

1) Handcrafted Feature Baselines: We establish classical baselines using FMA's official feature set (518 dimensions including MFCCs, spectral contrast, chroma, zero-crossing rate, tempo):

- **L2-Regularized Logistic Regression**: Linear model with cross-validated regularization strength $C \in \{0.01, 0.1, 1, 10, 50\}$
- **PCA + SVM (RBF)**: Dimensionality reduction followed by nonlinear SVM with grid-searched hyperparameters ($k \in \{64, 128, 256\}$, C, γ)

All features are standardized using z-score normalization before training.

2) CNN Baseline (FMA-small): A lightweight CNN for spectrogram classification:

- **Encoder**: 4 convolutional blocks with channel progression $1 \to 32 \to 64 \to 128 \to 256$
- **Each block**: 2 Conv layers (3×3), BatchNorm, ReLU, MaxPool (2×2), progressive dropout (0.10 $\to$ 0.25)
- **Pooling**: Global average pooling
- **Classifier**: Dense(256) $\to$ ReLU $\to$ Dropout(0.3) $\to$ Dense(num_classes)

Total parameters: 1.2M

3) CNN-Transformer Architecture: Our main architecture combines local feature extraction with global temporal modeling:

- **CNN Encoder** (segment-level): Four convolutional blocks with channel progression 32→64→128→256, each followed by batch normalization, ReLU activation, max pooling, and dropout (0.15). The encoder outputs a 256-dimensional embedding per segment.
- **Transformer Aggregator** (track-level): A baseline Transformer encoder with 2 layers and 4 attention heads, embedding dimension 256, feedforward dimension 1024, and dropout 0.1. A learnable `[CLS]` token aggregates evidence across all 8 segments. In addition to this

baseline, a larger configuration with embedding dimension 384, 3 Transformer layers, and 6 attention heads (384D/3L/6H) is also evaluated.

- **Classification Head**: Layer normalization followed by a linear projection (256 $\rightarrow$ `num_classes`).

Design rationale: CNN captures local spectro-temporal patterns (timbre, rhythm within 3-second windows), while Transformer attention enables adaptive weighting of different track regions based on genre relevance.

F. Training Configuration

All deep models are trained with:

- **Optimizer**: AdamW with learning rate 2×10^{-4}, weight decay 10^{-3}.
- **Scheduler**: Used the scheduler ReduceLROnPlateau (factor=0.5, patience=2).
- **Regularization**: Gradient clipping at 1.0, mixed-precision training (FP16).
- **Early stopping**: Patience 8 based on validation loss.
- **Batch size**: 16 (CNN-Transformer), 32-50 (CNN baseline).
- **Class imbalance (FMA-medium):** we evaluate both natural sampling and inverse-frequency weighted sampling. Some runs additionally use class-weighted losses.

All experiments use stratified train/val/test splits to preserve genre proportions.

IV. Experiments and Results

A. Research Questions

Our experiments are designed to answer the following questions:

1) **RQ1 (Dataset effect):** How does dataset quality and homogeneity (GTZAN vs. FMA) change which model family performs best?
2) **RQ2 (Handcrafted vs learned):** Do spectrogram-based CNNs outperform handcrafted feature baselines on noisy, diverse data?
3) **RQ3 (Normalization):** How do local vs. global normalization strategies affect optimization stability and final performance for the CNN-Transformer in Music Data?
4) **RQ4 (Scaling):** What changes when scaling from FMA-small (balanced) to FMA-medium (long-tailed, 16 classes)?
5) **RQ5 (Preprocessing ablations):** Which preprocessing choices (silent-segment filtering, augmentation strength, regularization) have the largest impact?

B. Pipeline Overview

The proposed pipeline for training Deep Models on the FMA dataset is given in the Pipeline Figure (Figure 1). The pipeline follows a hierarchical design. First, we convert raw audio into log-mel spectrograms, which provide a compact time–frequency representation that is widely used in audio recognition. We then segment each track into multiple fixed-length windows and apply strong augmentation during training. Segmentation increases the effective number of training examples and reduces GPU memory requirements, while also acting as a form of regularization by preventing the model from overfitting to a single, fixed excerpt of each song. Augmentations (time shifting, gain, noise, and SpecAugment) further encourage invariances that are desirable for genre recognition, such as robustness to small timing variations and recording conditions.

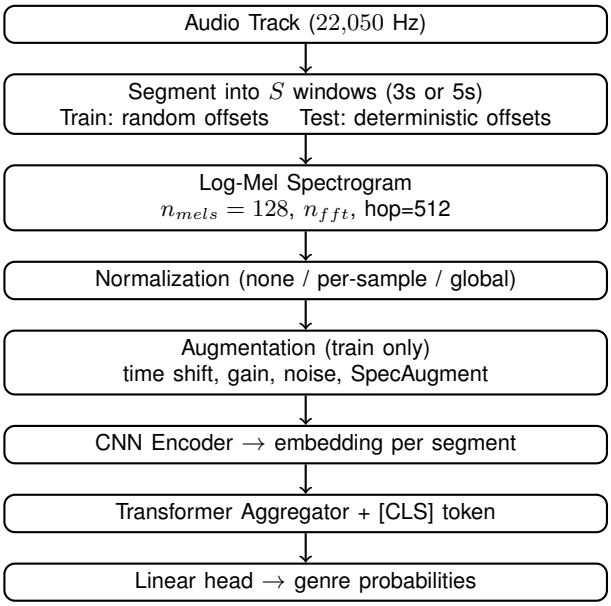

Fig. 1: End-to-end pipeline for segment-based genre classification.

Let $\{e_i\}_{i=1}^{S}$ be CNN embeddings for S segments and e_{cls} a learnable token. We form the sequence $E = [e_{cls}; e_1; \ldots; e_S]$ and apply a Transformer encoder. Self-attention is computed as:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

The final [CLS] representation is passed to a linear classifier to predict the genre.

C. Exploratory Data Analysis

1) Class Distribution: We analyze the class (genre) distributions for both FMA-small and FMA-medium to characterize label balance before training. FMA-small exhibits a relatively balanced distribution (Figure 2), with the largest class (Experimental) comprising only 13.26% of the dataset. In contrast, FMA-medium is substantially more imbalanced (Figure 3), with a max/min class frequency ratio of approximately 15:1. This degree of skew can bias learning toward majority genres and motivates mitigation strategies such as class weighting or balanced sampling.

2) PCA: To assess the separability of genres under a linear projection, we apply PCA to the precomputed `features.csv` representations for each subset. For FMA-small, the two-component PCA projection (Figure 4) explains only 19.5% of the variance and shows substantial overlap

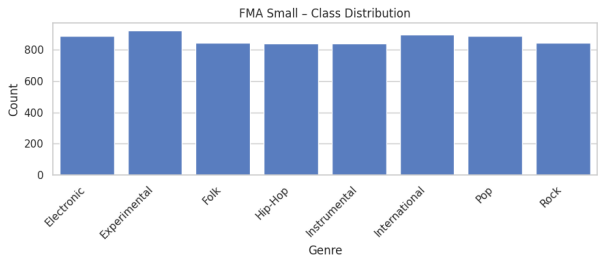

Fig. 2: Class distribution of the FMA Small dataset.

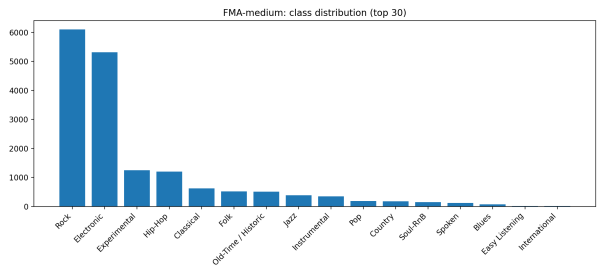

Fig. 3: Class distribution of the FMA Medium dataset.

between genres, indicating that discriminative structure is not well-captured by a low-dimensional linear subspace.

For FMA-medium, PCA is computed over $N{=}17000$ tracks with $D{=}518$ feature dimensions. The first two principal components explain 12.04% and 7.83% of the variance, respectively (19.87% total), again exhibiting heavy genre overlap (Figure 5). The consistently low explained variance in 2D and the strong inter-class mixing across both subsets suggest that genre boundaries are high-dimensional and likely nonlinear, motivating the use of expressive nonlinear models.

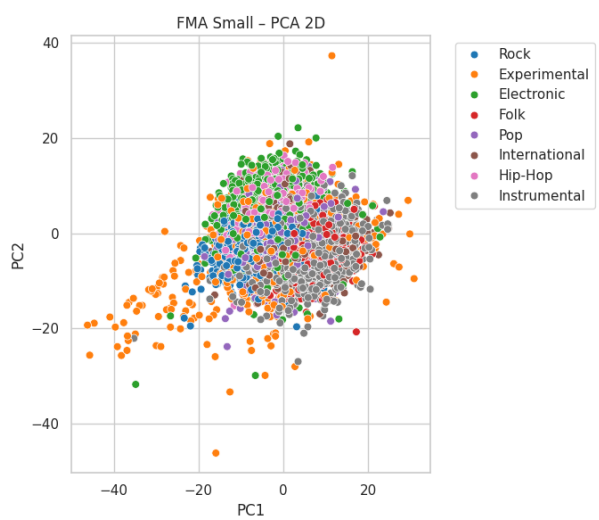

Fig. 4: Two-dimensional PCA projection of FMA Small features.

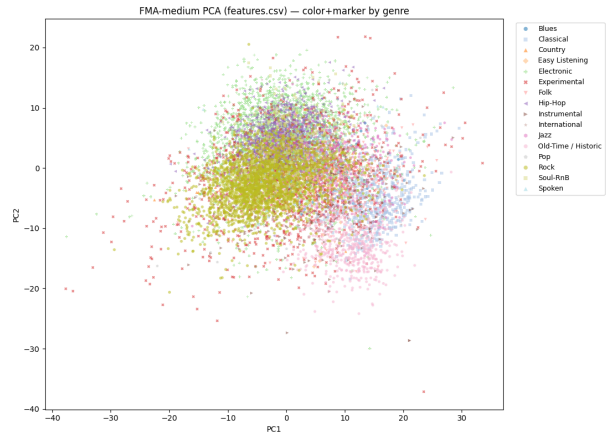

Fig. 5: Two-dimensional PCA projection of FMA Medium features.

D. Baseline Results: Handcrafted Features

We have our classical baselines using handcrafted audio descriptors from FMA and GTZAN.

1) FMA-small Results:

a) Logistic Regression: L2-regularized logistic regression achieves 53.7% test accuracy on FMA-small (Table I),

TABLE I: FMA Small — Logistic Regression Results

Class	Precision	Recall	F1
Electronic	0.60	0.64	0.62
Experimental	0.47	0.39	0.43
Folk	0.58	0.58	0.58
Hip-Hop	0.56	0.60	0.57
Instrumental	0.55	0.50	0.53
International	0.62	0.63	0.62
Pop	0.34	0.38	0.36
Rock	0.60	0.58	0.59
Accuracy		0.537	
Macro Avg	0.54	0.54	0.54

b) PCA + SVM (RBF): Adding nonlinear decision boundaries via RBF-kernel SVM improves performance to approximately 58% test accuracy with optimal configuration (128 PCA components, $C \approx 1$, $\gamma \approx 10^{-3}$). This suggests that the handcrafted features contain useful information, but they have little discriminative power.

2) GTZAN Results: GTZAN has stronger baseline performance due to more homogeneous data:

a) Logistic Regression: Achieves 72.9% test accuracy (Table II), 19% absolute improvement over FMA-small, highlighting the impact of data quality on linear model performance.

b) PCA + SVM (RBF): Achieves **91.9%** test accuracy with 58 PCA components, $C \approx 6.31$, $\gamma \approx 0.025$ (Table III). On clean data, handcrafted features with nonlinear classifiers can achieve very high accuracy.

c) Key Observation: The gap between FMA (58%) and GTZAN (92%) SVM performance proves that **data quality and curation matter more than the algorithm** for handcrafted

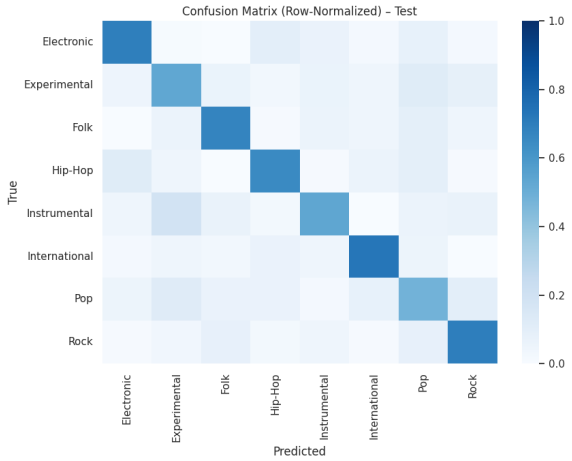

Fig. 6: Confusion matrix for PCA + SVM (RBF) on FMA Small.

TABLE II: GTZAN — Logistic Regression Results

Class	Precision	Recall	F1
Classical	0.91	0.94	0.93
Metal	0.78	0.85	0.81
Jazz	0.81	0.85	0.83
Pop	0.84	0.81	0.83
Accuracy		0.729	
Macro Avg	0.73	0.73	0.73

TABLE III: GTZAN — PCA + SVM (RBF) Test Results

Class	Precision	Recall	F1
Classical	0.90	0.95	0.92
Metal	0.97	0.99	0.98
Jazz	0.96	0.93	0.94
Hip-Hop	0.95	0.89	0.92
Accuracy		0.919	
Macro Avg	0.92	0.92	0.92

feature baselines. GTZAN's cleaner production and recording quality enable a highly optimal performance from classical methods.

E. Deep Learning Results

1) CNN Baselines: We evaluate CNNs on both datasets to test whether learned representations outperform handcrafted features.

a) FMA-small CNN: A lightweight CNN trained on full-track log-mel spectrograms achieves 59.1% test accuracy with an F1 score of 0.58 (Figure 7), outperforming all handcrafted feature baselines. This shows that learned spectro-temporal representations capture genre-discriminative patterns effectively.

b) GTZAN CNN: On GTZAN, the CNN achieves 75.0% test accuracy with the F1 score of 0.747 (Table IV). This performs worse than the SVM baseline (92%). This indicates that, on small and clean datasets, CNNs without detailed

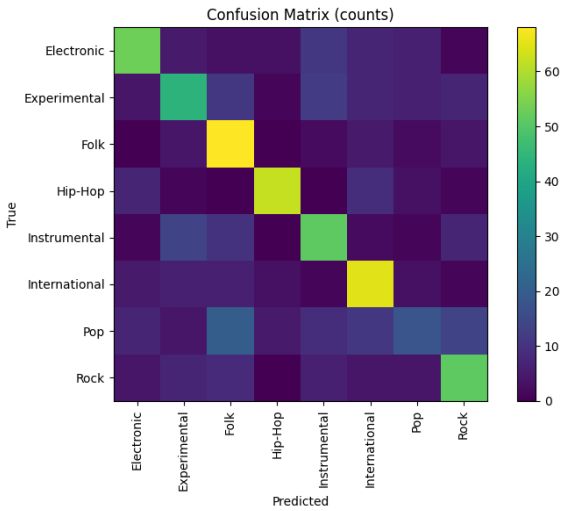

Fig. 7: FMA Small – CNN confusion matrix (59% accuracy).

augmentation or pretraining struggle to match well-optimized classical methods.

TABLE IV: GTZAN — CNN Classification Results (Test Set)

Class	Precision	Recall	F1
Disco	1.00	0.80	0.89
Hip-Hop	1.00	0.80	0.89
Pop	0.80	0.92	0.86
Metal	0.67	0.80	0.73
Accuracy		0.750	
Macro Avg	0.77	0.75	0.75

c) Key Insight: Data Size vs Model Complexity Trade-off: The contrasting CNN performance (FMA: beats SVM by 1%, GTZAN: loses to SVM by 17%) suggests:

- **Small clean data**: Handcrafted features + kernel methods win (GTZAN: SVM 92% CNN 75%)
- **Larger noisy data**: Learned representations win (FMA: CNN 59% SVM 58%)

This motivates our focus on FMA for deep learning experiments, where data scale enables effective representation learning.

2) CNN-Transformer: Normalization Study on FMA-small: We compare three normalization strategies using the CNN-Transformer architecture:

a) Global Normalization: 47.8% test accuracy with genuine multi-class discrimination (Table V). Best performing genres: Rock (F1=0.63), Hip-Hop (0.54), Electronic (0.49). Training dynamics show stable convergence without extreme volatility.

b) Per-sample (local) normalization (failure case): The per-sample normalization run produced an unusually low test accuracy (near single-class predictions). Test performance collapses to **11.38%** accuracy and near-zero Macro F1. The model predicts almost all samples as *International* (International recall ≈ 0.91), while the remaining genres have

near-zero recall. This indicates that local normalization can remove track-level energy/loudness cues and can destabilize optimization in small-data settings. Because this behavior is extreme, it may reflect an experimental or evaluation error.

TABLE V: FMA-small classification results with global normalization (CNN-Transformer).

Genre	Precision	Recall	F1-score
Electronic	0.74	0.58	0.65
Experimental	0.37	0.44	0.40
Folk	0.22	0.16	0.19
Hip-Hop	0.64	0.87	0.74
Instrumental	0.38	0.27	0.32
International	0.53	0.67	0.59
Pop	0.32	0.10	0.15
Rock	0.44	0.73	0.55
Accuracy		0.478	
Macro Avg	0.46	0.48	0.45
Weighted Avg	0.46	0.48	0.45

c) Key Finding: Importance of Loudness Preservation: The 47.8% accuracy shows that local normalization removes genre-relevant loudness information. Global normalization protects the differences in loudness between tracks and also provides scale consistency for gradient-based optimization. Accuracy dropped to 47.8% because transformers perform better on big datasets. FMA-small is a small-sized dataset for a transformer.

F. Scaling to FMA-medium

FMA-medium is a harder and more realistic setting than FMA-small. FMA-medium contains 16 genres and a strong class imbalance. For example, in the test split of our runs, *Rock* has 610 samples and *Electronic* has 532 samples, while *International* has only 2 samples and *Easy Listening* has 6 samples. Because of this imbalance, we report both **Macro F1** (class-balanced view) and **Weighted F1** (overall performance dominated by frequent classes).

1) Model and training setup: All FMA-medium experiments use the same high-level architecture: a CNN front-end applied to each audio segment, followed by a Transformer encoder that aggregates multiple segments from the same track.

- **Input representation:** log-Mel spectrogram in dB scale ($n_{mels} = 128$, $n_{fft} = 1024$).
- **Segmentation:** each track is represented by S segments of length 3s or 5s. During training, segment start positions are sampled randomly. During validation/test, start positions are evenly spaced.
- **Global normalization:** in some experiments, spectrograms are normalized by a dataset-level mean/std.
- **Sampling:** some runs use a weighted random sampler (inverse class frequency) to partially rebalance minibatches. Other runs use the natural (unweighted) sampling of the dataset.
- **SpecAugment:** time/frequency masking is applied with probability `aug_prob`.

TABLE VI: FMA-medium experiment summary (test set): Accuracy.

ID	Main change	Acc.
M0	Baseline model (3s, 256D/2L/4H, 8 seg/track)	0.689
M1	Global normalization + weighted random sampler	0.600
M2	M1 + head dropout 0.30 + label smoothing 0.05, 12 seg/track	0.665
M3	Larger backbone 384D/3L/6H + head dropout 0.25, 14 seg/track	**0.692**
A1	No silent-segment filtering (5s segments)	0.562
A2	Reduced regularization (attn dropout 0.05, larger batch)	0.554
A3	Long-tail tricks (sampler + weighted CE/focal)	0.565

TABLE VII: FMA-medium experiment summary (test set): Macro F1.

ID	Main change	Macro F1
M0	Baseline model (3s, 256D/2L/4H, 8 seg/track)	0.345
M1	Global normalization + weighted random sampler	0.347
M2	M1 + head dropout 0.30 + label smoothing 0.05, 12 seg/track	0.350
M3	Larger backbone 384D/3L/6H + head dropout 0.25, 14 seg/track	**0.394**
A1	No silent-segment filtering (5s segments)	0.321
A2	Reduced regularization (attn dropout 0.05, larger batch)	0.338
A3	Long-tail tricks (sampler + weighted CE/focal)	0.327

- **Silent-segment filtering (when enabled):** during training, very low-energy segments are avoided using a threshold (typically `−60 dB`) with resampling attempts.

2) Experiment naming: To make results easier to follow, we rename the most important experiments as: `M0`–`M3` for the main development line and `A1`–`A3` for negative results. Table VI–Table VIII summarize the outcomes.

3) M0: Baseline CNN-Transformer (our strong starting point): Our baseline configuration (`M0`) is already strong and reaches **68.9%** test accuracy (Macro F1=0.345, Weighted F1=0.657). This run uses 3-second segments, 8 segments per track, and a moderate Transformer backbone (256D, 2 layers, 4 heads). Even with this relatively small Transformer, the model learns good representations for frequent classes such as *Rock* and *Electronic*. However, it still struggles on rare classes, which is visible in the lower Macro F1 compared to Weighted F1.

a) Interpretation: This model is simple enough to train reliably, but it already gives competitive performance. So we treat it as the reference point for later improvements.

4) M1: Global-normalized + Sampler run: We retain a global-normalized reference configuration (`M1`) that reaches **60.0%** test accuracy (Macro F1=0.347, Weighted F1=0.637). In addition to enabling *global normalization*, `M1` is also the **first** run in which we activate a **weighted random sampler** (inverse-frequency sampling) to partially rebalance minibatches.

We use `M1` primarily as a *reproducible anchor*: its dataset-level normalization parameters and sampling policy are explicitly recorded, making it a stable baseline for subsequent tuning. Per-class behavior is typical for the long-tailed FMA-medium setting: performance is strong on frequent genres such as *Rock* (F1≈0.78), *Electronic* (F1≈0.74), and *Hip-Hop* (F1≈0.70), while extremely low-resource genres remain near zero F1 (e.g., *International*, *Easy Listening*, *Blues*).

TABLE VIII: FMA-medium experiment summary (test set): Weighted F1.

ID	Main change	Weighted F1
M0	Baseline model (3s, 256D/2L/4H, 8 seg/track)	0.657
M1	Global normalization + weighted random sampler	0.637
M2	M1 + head dropout 0.30 + label smoothing 0.05, 12 seg/track	0.649
M3	Larger backbone 384D/3L/6H + head dropout 0.25, 14 seg/track	**0.697**
A1	No silent-segment filtering (5s segments)	0.606
A2	Reduced regularization (attn dropout 0.05, larger batch)	0.597
A3	Long-tail tricks (sampler + weighted CE/focal)	0.610

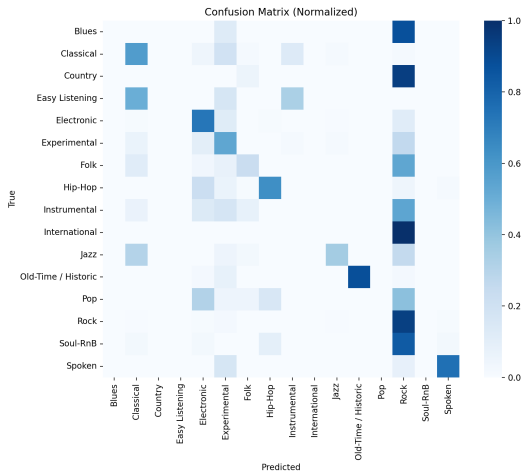

Fig. 8: (M0) Confusion matrix for the baseline CNN-Transformer (Acc.=0.689).

a) Interpretation: Overall, M1 is most useful as a stable reference because its dataset-level normalization parameters and sampling settings are fixed and documented. We use it as an anchor to evaluate whether later modifications (e.g., regularization, segmentation, or capacity changes) provide improvements relative to this standardized setup.

5) M2: Regularization and more segments (built on M1): In M2, we start from the global-normalized reference (M1) and add extra regularization (head dropout and label smoothing) while increasing the number of segments per track. This improves performance to **66.5%** accuracy (Macro F1=0.350, Weighted F1=0.649).

a) Interpretation: Increasing segments per track can help because the Transformer sees more diverse parts of each song. Also, mild regularization (dropout + label smoothing) can stabilize training in noisy datasets. However, M2 still does not surpass our strongest baseline M0, which suggests that other tuning choices (augmentation strength, data filtering, or sampling details) may be more important than these two changes alone.

6) M3 (best): Increasing capacity + head-only regularization: The best run (M3) increases the Transformer capacity (384-dimensional embeddings, 3 attention layers, 6 heads) and uses a simple but effective dropout only on the classifier head. It reaches **69.2%** test accuracy with the best Macro F1 (0.394) and Weighted F1 (0.697).

TABLE IX: FMA-medium classification results with Global Normalization (M2) + head dropout + label smoothing.

Genre	Precision	Recall	F1-score
Blues	0.00	0.00	0.00
Classical	0.53	0.37	0.44
Country	0.50	0.22	0.31
Easy Listening	0.00	0.00	0.00
Electronic	0.88	0.66	0.76
Experimental	0.37	0.41	0.39
Folk	0.41	0.21	0.28
Hip-Hop	0.71	0.66	0.68
Instrumental	0.17	0.28	0.21
International	0.00	0.00	0.00
Jazz	0.50	0.28	0.36
Old-Time / Historic	0.96	0.90	0.93
Pop	0.00	0.00	0.00
Rock	0.69	0.94	0.79
Soul-RnB	0.33	0.02	0.04
Spoken	0.32	0.58	0.41
Accuracy		0.665	
Macro Avg	0.40	0.35	0.35
Weighted Avg	0.67	0.66	0.65

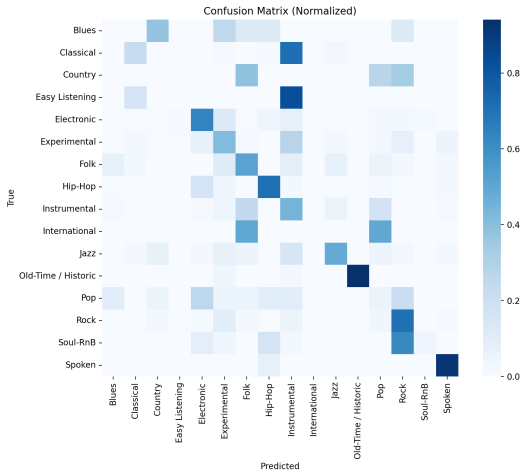

Fig. 9: (M1) Confusion matrix for the global-normalized reference model (Acc.=0.600).

a) Main takeaway: Compared to M0, the accuracy improvement is small (+0.3 points), but the Macro F1 improves much more (+4.9 points). This means the larger model helps the *smaller and medium-sized classes* more, which is important in long-tailed datasets.

7) Low performing experiments (A1–A3): What did not work well:

a) A1: No silent-segment filtering.: When silent-segment filtering is disabled, performance drops to **56.2%** accuracy (Macro F1=0.321). This supports the idea that very low-energy segments add noise and reduce training quality.

b) A2: Reduced regularization: Lowering attention dropout (and using a larger batch) reduces performance to **55.4%** accuracy. This suggests regularization is still important for generalization in this noisy dataset.

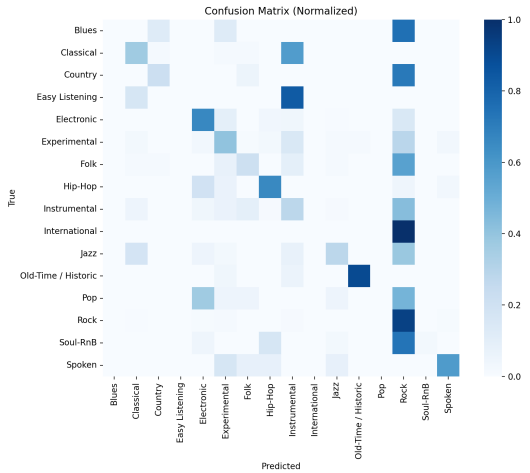

Fig. 10: (M2) Confusion matrix for M1 + head dropout + label smoothing (Acc.=0.665).

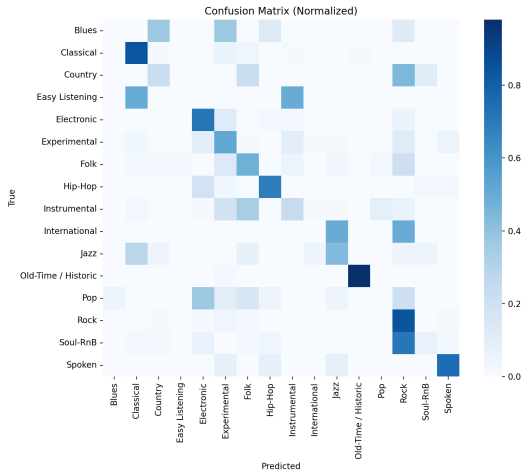

Fig. 11: (M3) Confusion matrix for the best configuration (larger backbone) (Acc.=0.692).

c) A3: Long-tail tricks: We also tried imbalance-focused losses/sampling (weighted CE / focal-like strategies). In our runs, these did not improve performance (Acc.=0.565) and were harder to tune. So for this project, the simplest approach (data filtering + good backbone + augmentation) was more reliable.

Table X reports the metrics per-class for M3. The strongest genres are *Old-Time / Historic* (F1≈0.98), *Rock* (0.83), *Electronic* (0.79), *Hip-Hop* (0.74), and *Classical* (0.72). However, some very low-resource classes remain unsolved (e.g., *International* and *Easy Listening*).

V. Discussion

A. Summary of Key Findings

Considering all experiments (handcrafted baselines, CNNs, and CNN-Transformer variants), we observed the following:

TABLE X: FMA-medium per-class results for our best CNN-Transformer model (M3).

Genre	Precision	Recall	F1
Blues	0.00	0.00	0.00
Classical	0.63	0.84	0.72
Country	0.18	0.22	0.20
Easy Listening	0.00	0.00	0.00
Electronic	0.88	0.72	0.79
Experimental	0.34	0.53	0.41
Folk	0.35	0.38	0.36
Hip-Hop	0.77	0.72	0.74
Instrumental	0.24	0.33	0.28
International	0.00	0.00	0.00
Jazz	0.47	0.44	0.45
Old-Time / Historic	0.98	0.98	0.98
Pop	0.00	0.00	0.00
Rock	0.82	0.85	0.83
Soul-RnB	0.17	0.07	0.10
Spoken	0.30	0.75	0.43
Accuracy		0.692	
Macro Avg	0.39	0.43	0.39
Weighted Avg	0.72	0.69	0.70

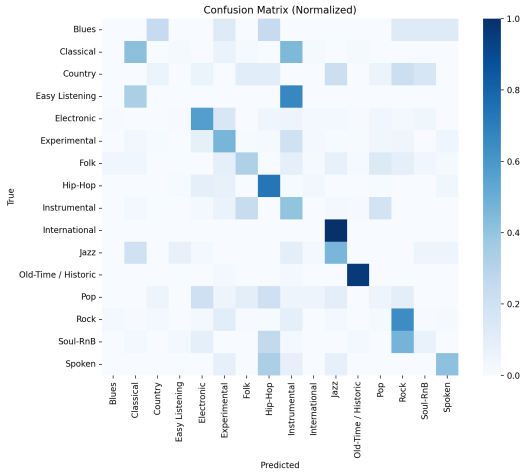

Fig. 12: (A1) Performance drop without silent-segment filtering (Acc.=0.562).

1) **Dataset quality and homogeneity strongly affect which method wins.** On GTZAN, handcrafted features combined with a nonlinear classifier perform extremely well (PCA+SVM reaches 0.919 accuracy), while our CNN reaches 0.750. This suggests that when a dataset is small and relatively clean, a well-tuned classical pipeline can be very hard to beat.
2) **On FMA-small, learned spectrogram features become competitive.** On FMA-small, the CNN trained on log-Mel spectrograms reaches 0.591 accuracy, slightly higher than PCA+SVM (0.58) and logistic regression (0.537). This supports the idea that in more diverse and noisier audio collections, learned representations can extract useful genre cues beyond handcrafted statistics.
3) **Normalization matters for the CNN-Transformer on**

TABLE XI: Accuracy-only comparison across datasets and model families.

Dataset	Method / Setting	Acc.
GTZAN	Logistic Regression (handcrafted)	0.729
GTZAN	PCA + SVM (RBF) (handcrafted)	**0.919**
GTZAN	CNN (log-Mel)	0.750
FMA-small	Logistic Regression (handcrafted)	0.537
FMA-small	PCA + SVM (RBF) (handcrafted)	0.580
FMA-small	CNN (log-Mel)	**0.591**
FMA-small	CNN-Transformer (global norm)	0.478
FMA-medium	`A1` No silent-segment filtering (5s seg)	0.562
FMA-medium	`A2` Reduced regularization	0.554
FMA-medium	`A3` Long-tail tricks (sampler + weighted CE)	0.565
FMA-medium	CNN-Transformer `M1` (global norm + sampler)	0.600
FMA-medium	CNN-Transformer `M0` (baseline tuned)	0.689
FMA-medium	CNN-Transformer `M2` (`M1` + specs)	0.665
FMA-medium	CNN-Transformer `M3` (best: 384D/3L/6H)	**0.692**

small-scale FMA. In the CNN-Transformer setup on FMA-small, global normalization produces meaningful multi-class discrimination (around 46–48% accuracy). In contrast, more local normalization strategies can remove loudness-related cues and can make optimization harder. Overall, we found that normalization is not just a technical detail, but part of the model design.

4) **Scaling from FMA-small to FMA-medium improves the CNN-Transformer, but the dataset remains long-tailed.** Moving to FMA-medium increases performance substantially. The best configuration (`M3`) reaches 0.692 test accuracy with Macro F1=0.394 and Weighted F1=0.697. The large gap between Macro and Weighted F1 shows that performance is dominated by frequent genres, while rare genres remain difficult.
5) **Silent-segment filtering is one of the most impactful preprocessing steps.** Disabling silent-segment filtering (`A1`) reduces accuracy from around 0.69 to 0.562 (Macro F1 drops to 0.321), confirming that very low-energy segments act like training noise and reduce genre-discriminative learning.

B. Cross-dataset Performance Comparison

Table XI summarizes our best results. For FMA-medium, we report **Macro F1** because the dataset is highly imbalanced and Macro F1 better reflects performance across all genres.

a) Why Macro F1 is important on FMA-medium: Weighted F1 values are much higher (e.g., `M3` has Weighted F1=0.697), because frequent genres dominate the metric. Macro F1 (0.394) reveals that rare genres still have low performance. This is consistent with the extreme test imbalance we observed (e.g., *International* has only 2 samples).

C. What Actually Improved FMA-medium Results

Among the main FMA-medium runs (`M0`–`M3`), the most consistent improvement came from **increasing model capacity while keeping regularization simple**. The best model (`M3`) increases the Transformer size (384D/3L/6H) and uses head dropout, which yields a small gain in accuracy relative to `M0`, but a larger gain in Macro F1. This suggests that extra capacity helps, especially for mid-frequency and smaller classes, even if the dominant classes already perform well.

On the other hand, explicit long-tail tricks (weighted losses, focal-style losses, sampling) did not help reliably in our experiments (`A3`), and were harder to tune. In practice, filtering low-information segments and using a strong backbone was more stable.

D. Limitations

Some genres have really few items. This makes it hard to track the learning of the model on these models and makes it unstable. Also, some genres are acoustically similar to humans as well. It is hard for the model to separate these genres because they overlap a lot. They are not strictly defined. Other than that, some genres need long segments to be identified. Because they have repeated motifs or a general progression type throughout the song. 3 and 5-second segments cannot always be enough to identify those genres.

E. Future Work

Based on our findings, the most promising improvements are:

- **Longer context aggregation:** increase the number of segments per track, or use hierarchical pooling.
- **More robust imbalance handling:** try moderate class-balanced sampling or two-stage head fine-tuning, instead of aggressive weighted/focal losses.
- **Pretraining or stronger augmentation:** pretrained audio encoders could improve FMA-medium, especially for rare genres.

References

[1] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.

[2] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "Fma: A dataset for music analysis," in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017.

[3] J. Bergstra, N. Casagrande, D. Erhan, D. Eck, and B. Kégl, "Aggregate features and AdaBoost for music classification," *Machine Learning*, vol. 65, no. 2-3, pp. 473–484, 2006. [Online]. Available: https://doi.org/10.1007/s10994-006-9019-7

[4] W. Zhang, W. Lei, X. Xu, and X. Xing, "Improved music genre classification with convolutional neural networks," in *Proceedings of the 17th Annual Conference of the International Speech Communication Association (INTERSPEECH 2016)*, San Francisco, CA, USA, September 2016, pp. 3304–3308. [Online]. Available: http://dx.doi.org/10.21437/Interspeech.2016-1236

[5] C. Liu, L. Feng, G. Liu, H. Wang, and S. Liu, "Bottom-up broadcast neural network for music genre classification," *Multimedia Tools and Applications*, vol. 80, no. 5, pp. 7313–7331, 2021.

[6] J. Shen and G. Xiao, "Music genre classification based on functional data analysis," *IEEE Access*, 2024.

[7] Y. Zhuang, Y. Chen, and J. Zheng, "Music genre classification with transformer classifier," in *Proceedings of the 2020 4th international conference on digital signal processing*, 2020, pp. 155–159.

[8] N. Ndou, R. Ajoodha, and A. Jadhav, "Music genre classification: A review of deep-learning and traditional machine-learning approaches," in *2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*, 2021, pp. 1–6.

[9] S. Oramas, O. Nieto, F. Barbieri, and X. Serra, "Multi-label music genre classification from audio, text, and images using deep features," 2017. [Online]. Available: https://arxiv.org/abs/1707.04916

[10] H. Bahuleyan, “Music genre classification using machine learning techniques,” 2018. [Online]. Available: https://arxiv.org/abs/1804.01149
[11] N. Pelchat and C. M. Gelowitz, “Neural network music genre classification,” *Canadian Journal of Electrical and Computer Engineering*, vol. 43, no. 3, pp. 170–173, 2020.
[12] R. Gan, T. Huang, J. Shao, and F. Wang, “Music genre classification based on VMD-IWOA-XGBOOST,” *Mathematics*, vol. 12, no. 10, p. 1549, 2024. [Online]. Available: https://doi.org/10.3390/math12101549

Appendix: Author Contributions

Yunus Emre Ulusoy: Designed and implemented the CNN-Transformer architecture, conducted model training runs on FMA datasets, performed experiments, and generated all plots and confusion matrices.

Semih Doğan: Designed and implemented the CNN-Transformer architecture, conducted model training runs on FMA datasets, performed experiments, and generated all plots and confusion matrices.

Alp Eren Gül: Conducted literature review, read and analyzed research papers on music genre classification, mixed findings from classical and deep learning approaches.

Burak Ala: Conducted literature review, read and analyzed research papers on music genre classification, mixed findings from classical and deep learning approaches.

Umut Köprülü: Wrote the introduction and conclusion sections, explained the motivations, and the general project outline. Worked on the data preprocessing pipeline.